

UNIVERSITATEA DIN BUCUREȘTI
FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ
DEPARTAMENTUL CALCULATOARE ȘI TEHNOLOGIA INFORMAȚIEI

PROIECT
METODE DE DEZVOLTARE SOFTWARE

PROFESOR COORDONATOR PROIECT:

CEAUȘESCU CIPRIAN-MIHAI

STUDENȚI:

BĂLAN MIHAI

OLARU BIANCA ELENA

BUCUREȘTI

2026

UNIVERSITATEA DIN BUCUREȘTI
FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ
DEPARTAMENTUL CALCULATOARE ȘI TEHNOLOGIA INFORMAȚIEI

**SISTEM DE ANALIZĂ A CONȚINUTULUI
UNEI CĂRȚI**

PROFESOR COORDONATOR PROIECT:

CEAUȘESCU CIPRIAN-MIHAI

STUDENȚI:

BĂLAN MIHAI

OLARU BIANCA ELENA

BUCUREȘTI

2026

CUPRINS

1. Introducere.....	4
1.1 Tema aleasă.....	4
1.2 Descriere.....	4
1.3 Cerințele și specificațiile proiectului.....	4
1.4. Limbajul de programare ales.....	5
2. Turul aplicației dezvoltate.....	8
2.1 Meniul principal.....	8
2.2 Pagina cărți analizate.....	9
2.3 Analiza documentului.....	9
3. Puncte slabe, soluții și probleme deschise.....	14
3.1 Probleme generale.....	14
3.2 Soluții.....	14
3.3 Probleme deschise.....	15
4. Concluzii.....	17

1. Introducere

1.1 Tema aleasă

Tema aleasă de noi, “Sistem de analiză a conținutului unei cărți” urmărește interpretarea nu printr-o manieră analitică obiectivă a lucrărilor literare (comparații cu alte scrieri, note critice, simbolistică și context istoric, ș.a.m.d.) ci prin analiza directă a conținutului. Astfel, ne dorim a transmite faptul că tehnica actuală ajută enorm chiar și atunci când este vorba despre domenii socioumane, iar statisticile furnizate de aplicația în lucru relevă cu adevărat informații pe care un studiu aprofundat doar le întărește.

1.2 Descriere

Ne dorim să implementăm o aplicație care primește un roman într-un format *.pdf* și furnizează, după analiza acestuia, statistici relevante despre acesta: câte dialoguri există între personajele de diferite sexe existente în roman, câte personaje sunt, ce statut au, în ce manieră este scris romanul în funcție de modurile și timpurile verbale existente, un graf între toate personajele romanului unde se văd pe arce incidențele dialogurilor dintre personaje, ș.a.m.d.

1.3 Cerințele și specificațiile proiectului

Pentru a asigura succesul sistemului, am stabilit următoarele cerințe:

1. Procesarea Documentelor:

- Sistemul trebuie să permită încărcarea fișierelor de tip *.pdf*.
- Extracția textului brut și curățarea acestuia (recunoașterea diacriticelor din alfabetul limbii române, curățarea fundalului de obicei îngălbenit din cauza vechimii cărților scanate).

2. Identificarea Entităților:

- Identificarea automată a personajelor și atribuirea unui sex/gen pe baza contextului și a numelui.
- Clarificarea rolului personajului (principal, episodic, secundar).

3. Analiza Dialogului și a Relațiilor:

- Extragerea replicilor și actualizarea bazei de date.

- Calcularea frecvenței interacțiunilor prin update-uri în baza de date pentru generarea ulterioară a grafului, care reprezintă o diagramă a relațiilor dintre personaje.

4. Vizualizarea Datelor:

- Generarea unui graf social interactiv (diagrama relațiilor).
- Interogarea bazei de date asupra personajelor/ interacțiunilor lor.
- Generarea unui rezumat foarte scurt al cărții.
- Interogarea unui alt model AI care oferă răspunsuri la întrebările adresate comportându-se ca un critic.

Specificații:

- **LLM 1 (Extracție):** Utilizarea unui model de LLM pentru parsarea textului dens și identificarea nuanțată a dialogurilor (cine cui vorbește).
- **LLM 2 (Analiză):** Utilizarea unui model LLM pentru generarea rezumatului descărcabil în interfață.
- **LLM 3 (Interogare):** Utilizarea unui alt model care oferă răspunsuri într-un textbox privind diversele întrebări ce privesc personaje
- **Backend:** API bazat pe **Django REST Framework** pentru gestionarea datelor (PostgreSQL).
- **Frontend:** Interfață web care folosește API-ul, **Vis.js** care ne ajută să vizualizăm graful final al relației dintre personaje.

1.4. Limbajul de programare ales

Aplicația este implementată pe baza unei arhitecturi decuplate, utilizând următoarele limbaje de programare și framework-uri:

1. Backend (Python & Django REST Framework):

- **Python:** Limbajul de programare principal utilizat pe server datorită ecosistemului bogat în biblioteci de inteligență artificială (SDK-ul Google Gemini) și procesare de text (Tesseract OCR pentru conversia PDF-urilor și python-docx pentru generarea rapoartelor).
- **Django:** Framework-ul web utilizat pentru structurarea proiectului și gestiunea bazei de date relaționale prin intermediul ORM-ului (Object-Relational Mapping).
- **Django REST Framework (DRF):** Utilizat pentru construirea unei interfețe de programare a aplicației (API REST) robuste, care asigură comunicarea asincronă și transferul de date (în format JSON) între backend și frontend.

2. Gestiunea Datelor (PostgreSQL & Docker):

Pentru stocarea persistentă a metadatelor cărților, personajelor și a relațiilor identificate, se utilizează sistemul de gestiune a bazelor de date relaționale **PostgreSQL**, rulat într-un container izolat prin **Docker Compose**.

3. Frontend (HTML5, CSS3 & JavaScript/Vis.js):

- Interfața utilizatorului este de tip **SPA** (Single Page Application), construită folosind tehnologii web fundamentale (HTML5 pentru structură și CSS3).
- Dinamica aplicației, controlul stărilor de analiză și comunicarea cu API-ul sunt realizate în **JavaScript**.
- Vizualizarea interactivă a grafului de relații dintre personaje este generată dinamic pe un element Canvas cu ajutorul bibliotecii dedicate **Vis.js** (modulul vis-network).

4. Arhitectura software și Șabloane de Proiectare (Design Patterns):

Arhitectura MVC (Model-View-Controller):

Aplicația respectă șablonul arhitectural MVC, adaptat pentru o structură decuplată (REST API):

- **Model:** Reprezentat de modelele Django (models.py) și baza de date PostgreSQL, care definesc schema datelor pentru cărți, autori, personaje și relații.
- **View (Interfața):** Reprezentată de componenta frontend (Single Page Application în HTML5/CSS3), care se ocupă exclusiv cu afișarea datelor și randarea grafului interactiv.
- **Controller (Controlerul):** Reprezentat de rutele (urls.py) și vederile Django (views.py). Acestea recepționează cererile de la frontend, coordonează logica de business, pornesc thread-urile de analiză în fundal și returnează răspunsurile în format JSON.

Șablonul Singleton (în contextul multithreading-ului):

Șablonul de proiectare Singleton nu este aplicat la nivelul întregii aplicații globale, ci este utilizat în mod specific în mecanismul de gestionare a execuției asincrone (threading). Sistemul utilizează un blocaj unic de tip mutex (threading.Lock) și o structură unică de monitorizare a stării execuției pentru a orchestra thread-urile din fundal. Acest comportament de Singleton garantează că pentru fiecare carte încărcată există un singur flux activ de procesare OCR sau analiză AI, prevenind problemele de concurență (race conditions) la nivelul bazei de date și garantând consistența progresului raportat către utilizator.

2. Turul aplicației dezvoltate

2.1 Meniul principal

Pagina care reprezintă **meniul principal** al proiectului ne permite să încărcăm documentul pe care ne dorim să îl scanăm și analizăm, în format **.pdf**. Tot aici putem vedea progresul paginilor care au fost scanate.

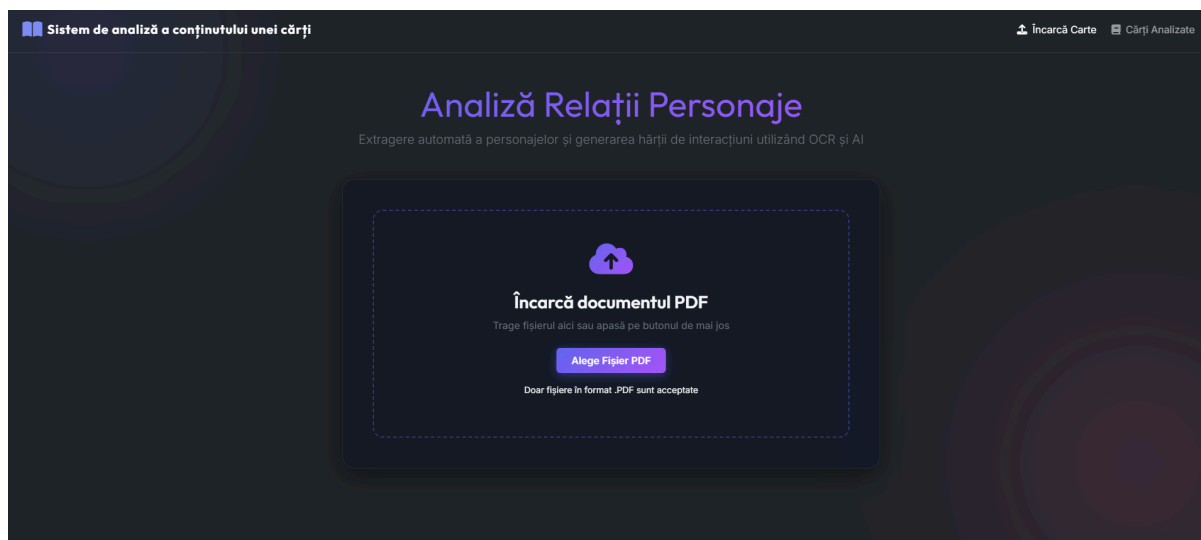


Fig. 1 – Meniul principal



Fig. 2 – Parsarea cărții

2.2 Pagina cărți analizate

Documentele introduse în meniul principal sunt ulterior stocate în baza de date implementată cu Docker, astfel că acestea sunt reținute de aplicație și pot fi accesate într-o nouă pagină, unde se vor regăsi **toate cărțile scanate**.

Aici, pentru fiecare carte, putem vedea date precum titlu, autor, numărul de pagini și anul publicării. Aplicația ne permite să ștergem toate documentele scanate, să le ștergem individual, să le modificăm titlul sau să accesăm analiza fiecăreia.

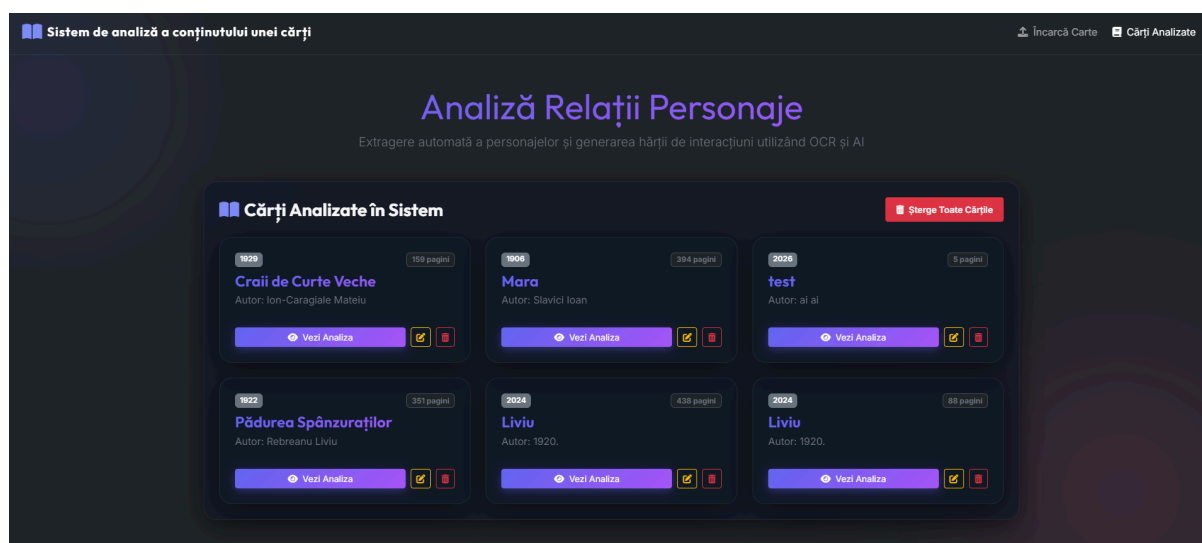


Fig. 3 – Pagina cărților analizate

2.3 Analiza documentului

În meniul de analiză individuală a documentului există o serie de instrumente implementate care să ne ajute să ajungem la rezultatul dorit:

- **Analizează relații:** generează informații și o **hartă interactivă a relațiilor dintre personaje** sub forma unui **graf**, în care nodurile sunt personajele, iar muchiile sunt relațiile între acestea (dacă există);
- **Generează rezumat:** returnează datele sub o formă bine structurată, folosind JSON;
- **Șterge date analiză:** curăță baza de date locală pentru a reîncepe scanarea de la zero;
- **Editează date:** permite utilizatorului să corecteze manual datele cărții (titlul/autorul);
- **Descarcă rezumat Word:** preia structura JSON generată de agentul și o compilează într-un document .docx;

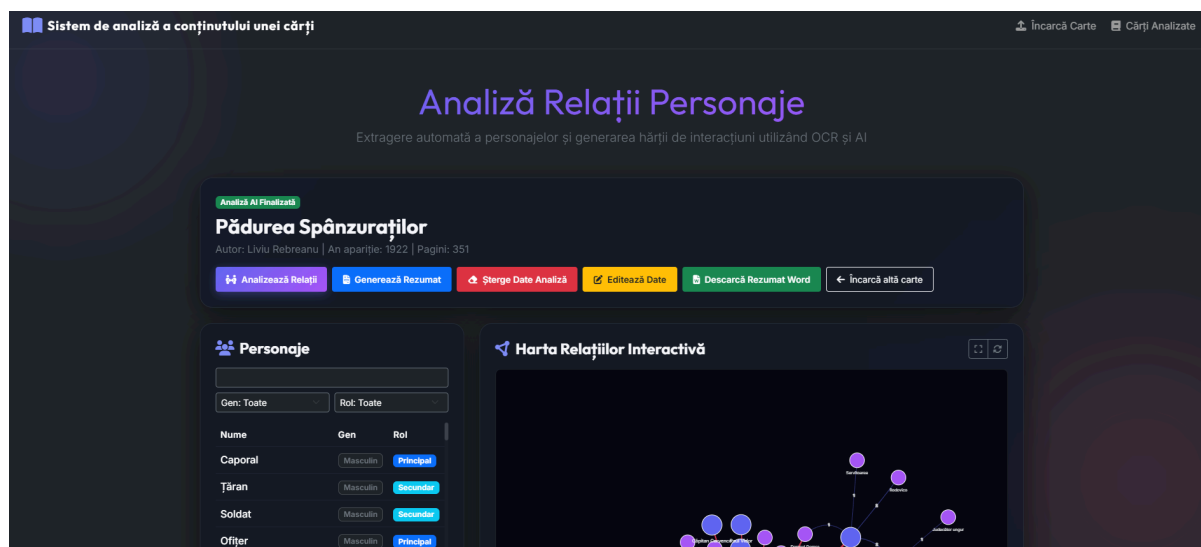


Fig. 4 – Analiza cărții „Pădurea Spânzuraților”

Pentru **analiza relațiilor**, agentul se folosește de **parsarea cărții** pentru a determina informații despre personaje și relațiile între acestea. Deși ne-am fi dorit ca analiza să transmită și tipul exact de relație dintre personaje, ne-am dat seama în timpul testelor că acest lucru diferă foarte mult în funcție de stilul autorului, de ironii, metafore și subtilități, de la carte la carte. Din acest motiv, ca să nu avem erori de interpretare, ne-am limitat la a crea un **graf** care ne indică prin **muchii** existența relației între două personaje. Această relație apare în sistem dacă între personaje a avut loc **cel puțin un dialog** pe parcursul cărții, aplicația numărând practic de câte ori au interacționat.

În interfață, am vrut ca toate aceste date să fie ușor de urmărit. Astfel, putem, printr-un simplu clic, să **evidențiem un personaj anume și relațiile sale directe** cu celelalte personaje, lucru care curăță ecranul și ne ajută să vedem imediat rețeaua lui de contacte. Totodată, utilizatorul are la dispoziție un **filtru de căutare** prin care să identifice mai ușor personajele în funcție de trăsăturile lor (masculin/feminin, principal/secundar/episodic etc.).

Pe lângă grafic, am adăugat și un **submeniu** unde se poate vedea exact **câte dialoguri au avut loc între toate personajele** (două câte două), sub forma unei liste clare de statistici. În acest submeniu am ales să integrăm încă un **agent** inteligent (bazat pe API-ul Gemini) căruia i se pot pune mai multe întrebări libere despre personaje. Acest al doilea agent este conectat direct la baza noastră de date, așa că el combină ce știe deja despre carte cu datele extrase de noi (cât de des vorbesc personajele, ce statut au) ca să ofere răspunsuri cât mai exacte în chat.

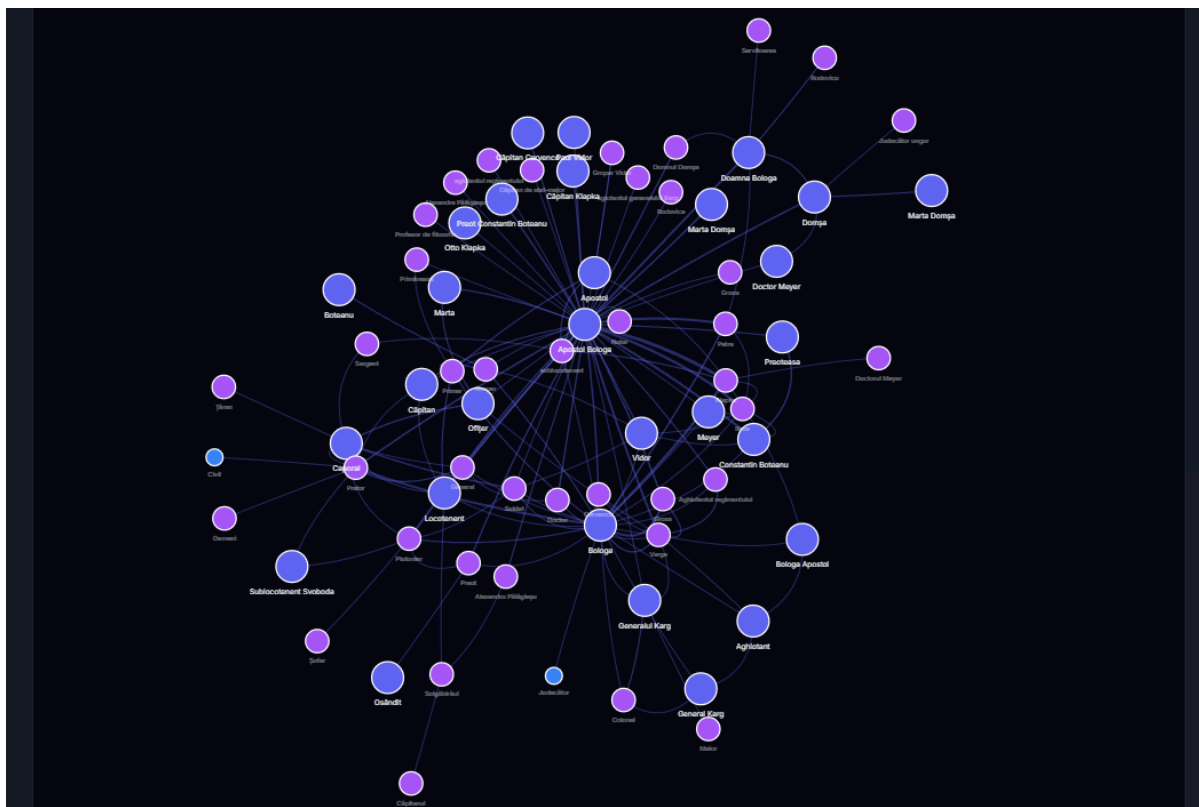


Fig. 5 – Harta interactivă a relațiilor (sub formă de graf)

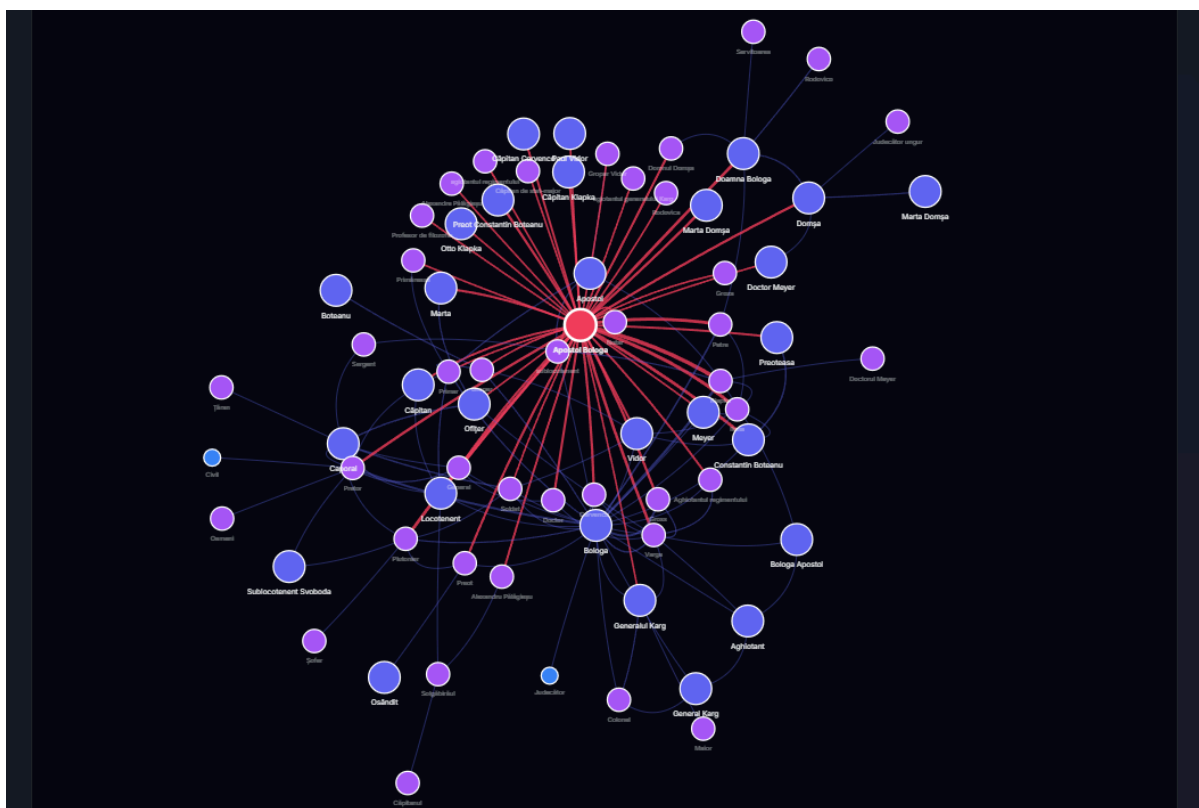


Fig. 6 – Evidențierea unui personaj și a relațiilor sale printr-un clic



Fig. 7 – Filtru de căutare a personajelor (și a relațiilor acestora)

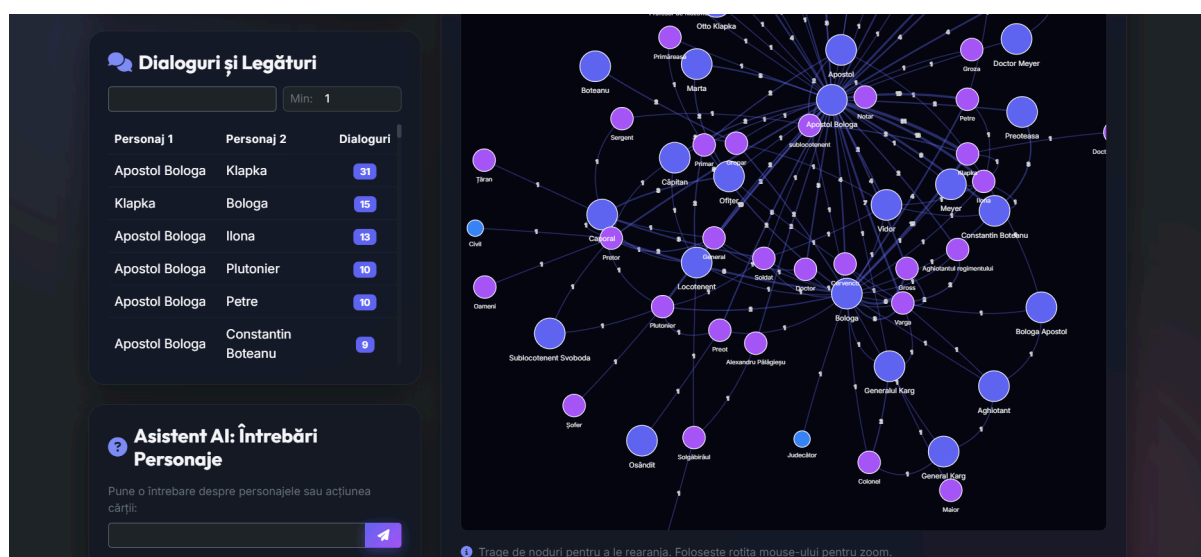


Fig. 8 – Submeniu și graful relațiilor

Cel de-al treilea agent implementat se ocupă exclusiv de citirea textului complet și de generarea rezumatelor. Agentul Gemini ne returnează datele într-o structură foarte curată și fixă (folosind o schemă JSON). Din acest motiv, ecranul de rezumat este împărțit în trei părți mari:

- **Rezumatul General:** O sinteză clară și cursivă a întregului roman (limită de maximum 3-4 paragrafe), care prezintă conflictul central și cum evoluează acțiunea.
- **Temele Principale:** O listă cu cele mai importante idei sau motive literare pe care AI-ul le identifică în text (cum ar fi destinul, datoria, familia, războiul).

Pentru acest proces am adăugat mecanisme de siguranță în spate: dacă apar probleme cu internetul sau dacă cheile API gratuite își ating limita de cereri, agentul nu blochează aplicația. El trece automat pe un mod de simulare care oferă un rezumat standard bine scris, asigurându-ne astfel că interfața rămâne stabilă și funcțională în timpul prezentării.

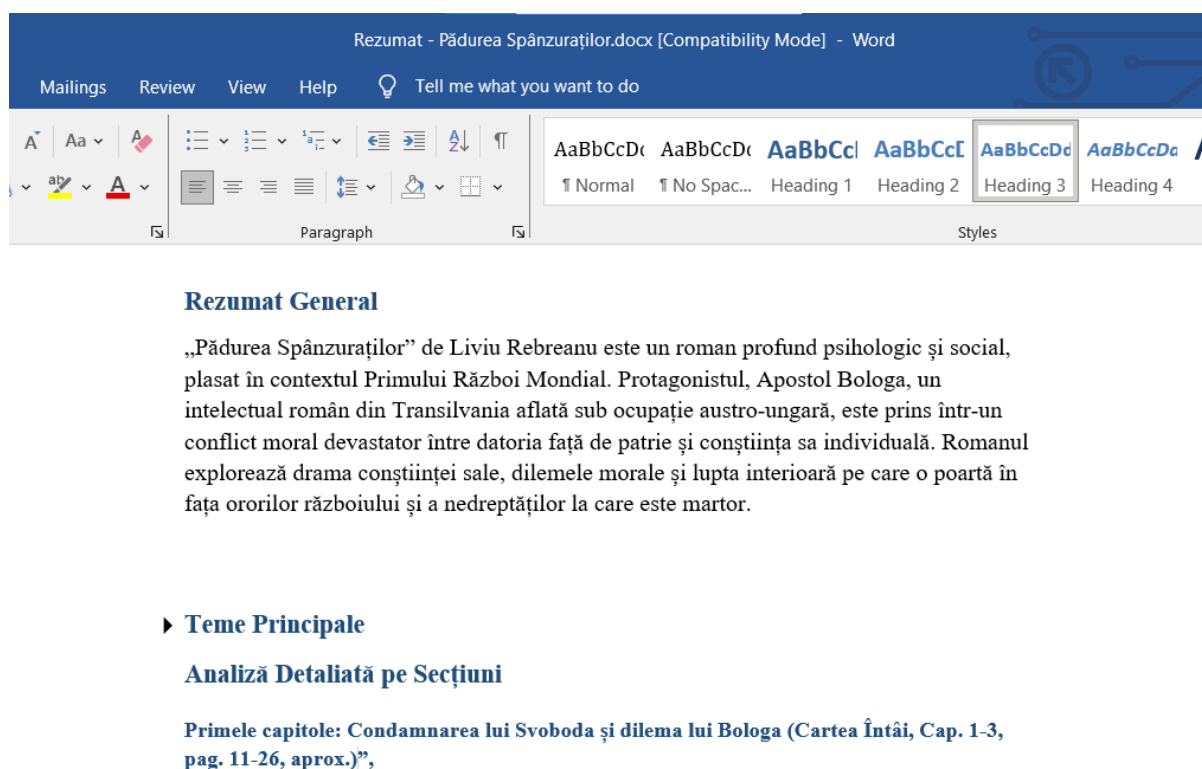


Fig. 9 & 10 – Extras din rezumatul .docx generat

3. Puncte slabe, soluții și probleme deschise

3.1 Probleme generale

Concluzia la care am ajuns în urma realizării aplicației ne demonstrează faptul că operele din limba română ne dau mari bătaii de cap, cu atât mai mult cu cât romanele studiate în implementare sunt în formate mai vechi.

O interpretare a datelor furnizate s-ar putea face generic doar ca o fotografie finală atunci când un cititor – fie el și experimentat – ajunge să citească cartea și să-i deprindă sensurile. Graficul rezultat funcționează ca o demonstrație sau o contradicție pentru ceea ce cititorul observă, ajutându-l să observe și să deducă mai ușor detalii precum **planurile narative și relațiile dintre personaje**. În cazul romanului “Ion”, de exemplu, incidența cea mai mare o au nodurile specifice lui Titu Herdelea și Ion Glanetașu - demonstrează cele două planuri narative din carte, deci personajele în jurul cărora acestea se conturează. Pentru o operă cum ar fi “Craii de Curtea-veche”, același prompt transmis agentului AI îl face să identifice două personaje diferite, care împărtășesc totuși majoritar relații cu aceleași persoane: Apostol Bologa și Bologa. În contextul cărții sunt unul și același, dar interpretarea apare așa datorită puternicilor conflicte interioare și a monologului lui Bologa apărut în carte. Deci apariția ciudată a aceluiași personaj are o posibilă interpretare - dar tot pentru un cunoscător.

Pentru un text simplu, AI-ul realizează graficul perfect, atunci când și documentul este de o calitate superioară. Am putea spune că pentru calitatea materialului pe care îl introducem aplicației, aceasta din urmă ne furnizează un rezultat valid care confirmă chestiuni enunțate anterior de critici sau profesori. Aplicația vine, deci ca o demonstrație matematică a unor concepte literare deja existente, dar interpretarea este anevoioasă dacă lucrarea nu a fost citită de utilizator înainte. Se pot face totuși supoziții: ce presupun că se va întâmpla din punct de vedere a dinamicii personajelor înainte să citesc cartea și ce se întâmpla de fapt? / s-a întâmplat exact ceea ce am citit?

3.2 Soluții

Cu minime cunoștințe de programare se poate face o pagină de interpretare a datelor, dar nu pentru un roman anume, căci acestea diferă. Nu există un prompt general sau o

formulă invariabilă care să interpreteze aceste grafuri fără o intervenție subiectivă, deci aplicația are un potențial uriaș din acest punct de vedere.

Prompturile deja existente se pot îmbunătăți și s-ar putea compara rezultatul generat de un prompt cu unul al altuia, iar soluția optimă să fie păstrată. Acest lucru ar crea un randament mult mai mare al aplicației și graful generat ar fi mai aproape de adevăr. De asemenea, rezumatul ar putea fi făcut mult mai amplu, poate pe capitole chiar, dar structura romanelor diferă, implicit împărțirea acestora, motiv pentru care am ales să formulăm un rezumat mai general. Am fi putut alege ca temă “Analiza conținutului unor romane care sunt împărțite pe capitole”, dar deja studiul ar fi fost nișat și nu se potrivea cu demersul nostru inițial.

3.3 Probleme deschise

În urma analizei comportamentului aplicației și a limitărilor identificate în procesarea operelor literare complexe, au fost conturate următoarele probleme deschise și direcții de dezvoltare viitoare:

1. Rezolvarea coreferințelor psihologice și a dedublării personajelor:

O problemă deschisă o reprezintă optimizarea algoritmilor LLM pentru a face diferența între personaje fizice distincte și instanțe ale aceluiași personaj apărute în monologuri interioare sau stări de dedublare psihologică (cum este cazul lui Apostol Bologa). O direcție viitoare ar fi implementarea unui modul de validare semantică (post-procesare) care să unifice aceste entități pe baza contextului narativ.

2. Adaptabilitatea dinamică a prompturilor:

Deoarece structura, stilul și vocabularul romanelor diferă considerabil în funcție de epocă și autor, o singură formulă de promptare nu este universal optimă. Dezvoltarea unui sistem hibrid care analizează **preliminar** stilul cărții (ex: română veche vs. română modernă) și selectează automat cel mai potrivit prompt adaptat reprezintă o direcție importantă de cercetare.

3. Segmentarea structurală inteligentă a operelor:

Extinderea rezumatelor la un nivel granular (pe capitole sau părți) fără a limita utilizarea aplicației doar la cărți cu structură fixă. Acest lucru presupune dezvoltarea unui

algoritm capabil să detecteze automat în mod euristic delimitările dintre capitole (indiferent de modul în care autorul le-a numerotat sau denumit) și să trimită loturi specifice către modelul de rezumare.

4. Interpretarea automată a grafului pentru utilizatorii naivi:

Pentru a elimina bariera de înțelegere în cazul utilizatorilor care nu au citit opera în prealabil, o direcție viitoare este implementarea unui agent AI explicativ. Acesta ar analiza structura topologică a grafului rezultat (centralitatea nodurilor, comunitățile formate, grosimea muchiilor) și ar genera o explicație textuală coerentă despre dinamica acțiunii și planurile narative evidențiate în graf.

4. Concluzii

Dezvoltarea acestui **sistem de analiză a conținutului unei cărți** ne-a arătat cum tehnologiile moderne de inteligență artificială generativă pot fi aplicate cu succes în analiza textelor literare. Integrarea API-ului Gemini prin agenți specializați a rezolvat probleme dificile de interpretare a contextului, cum ar fi identificarea personajelor ascunse în spatele pronumelor sau corectarea poreclelor.

Deși în faza inițială a proiectului ne propusesem să realizăm și o analiză lingvistică bazată pe modurile și timpurile verbale, pe parcursul dezvoltării am decis să pivotăm și să ne concentrăm toate resursele pe optimizarea analizei de grafuri și pe implementarea sistemului de **chatbot (Q&A)**. Am considerat că **harta interactivă a relațiilor** și posibilitatea de a pune întrebări libere aduc o valoare adăugată mult mai mare pentru utilizatorul final decât o simplă numărare statistică a verbelor.